

**Parcial 1. Liquidación semanal de una
empresa de mensajería**

**Sebastian Solorzano Garcia
Juan David Cardona Petrel
Nikoll Morales Londoño**



Profesor: Julián Esteban Gutiérrez Posada

**Universidad del Quindío
Facultad de Ingeniería
Programa Ingeniería de Sistemas y Computación
Programación III
Armenia, Octubre de 2026**

Parte A Diseño de la Solución

A.1 Estructura Modular y Separación de Responsabilidades

El proyecto se diseñó de forma modular respetando estrictamente el principio de separación de responsabilidades y la distinción entre funciones puras e impuras:

Módulo	Responsabilidad	Tipo de Función
principal.exs (Interfaz)	Punto de entrada de la aplicación. Carga todos los archivos del proyecto, define la lógica de interacción en el módulo Interfaz, captura entradas del usuario (servicios adicionales, código de repartidor), coordina la generación de reportes y emite la salida.	Impuro
Datos	Almacena y provee los datos estáticos iniciales de la empresa (repartidores con indicador de bicicleta, zonas con área geográfica y la lista completa de servicios semanales válidos e inválidos).	Puro
Validaciones	Implementa las reglas de negocio para validar la consistencia de cada servicio individual y clasificar los servicios entre válidos y rechazados.	Puro
Liquidacion	Calcula tarifas, ajustes por puntualidad, bonificaciones	Puro

Módulo	Responsabilidad	Tipo de Función
	por kilometraje diario, alquileres de transporte y el neto final a pagar por repartidor	
Reportes	Genera los análisis operativos de R1 a R8, ranking dinámico, integración con empresas aliadas y las mediciones de desempeño temporal.	Puro
Util / Util2	Provee funciones auxiliares de propósito general para entrada y salida de datos, procesamiento de colecciones y conversión de sus elementos a texto.	Impuro
pruebas.exs	Script auxiliar ejecutable para pruebas unitarias e integración rápida. Permite validar los datos de entrada y ejecutar aisladamente funciones específicas.	impuro

A.2 Elección de Colecciones y Estructuras de Datos

- **Listas (Lists):** Estructura principal para almacenar colecciones secuenciales (servicios sin validación, servicios válidos, servicios rechazados, repartidores y zonas) Preservan el orden uno a uno en el mismo orden en que fueron ingresados.
- **Mapas (Maps):** Estructura principal para representar entidades individuales y sus atributos mediante átomos (:codigo, :nombre, :bicicleta, :id, :area, :repartidor, :zona, :dia, :kilometros, :retraso) permitiendo acceso directo. También se utilizan como

acumuladores.

- **Tuplas (Tuples):** Utilizadas como claves compuestas en agrupación {repartidor, dia} y para asociar los servicios rechazados con su motivo específico de falla {servicio, :repartidor_desconocido} o retornos con resultado {:ok, servicio}.
- **Keyword Lists:** Implementadas para configurar opciones opcionales en funciones de ordenamiento y filtrado como ranking [orden: :desc, limite: 5].

A.3 Estructuras de Control e Iteraciones

- **with:** Utilizado en Validaciones.validar_servicio para encadenar las cinco reglas de negocio en el orden exacto requerido. Se detiene inmediatamente en la primera regla que falle y retorna la tupla con el motivo específico.
- **cond:** Utilizado en Liquidacion.factor_puntualidad para evaluar los rangos continuos de retraso (-30 a 0 min → 1.08; 1 a 10 min → 1.00; 11 a 30 min → 0.90; >30 min → 0.75)
- **if/case:** Utilizados para condicionales binarias (bonificación por kilometraje ≥ 80 km, cobro de alquiler de bicicleta, verificación de metas).
- **Funciones de Enum:** Se emplearon exclusivamente funciones declarativas del módulo Enum (map, reduce, group_by, split_with, filter, sort_by, all?, any?, frequencies) para la transformación, filtrado y agregación de colecciones.

A.4 Función ranking mediante Keyword Lists

La función de ranking permite parametrizar dinámicamente el ordenamiento y el límite de elementos presentados:

```
def ranking(reporte, opciones) do
  orden = Keyword.get(opciones, :orden, :desc)
  limite = Keyword.get(opciones, :limite, length(reporte))
  orden = if orden == :asc, do: :asc, else: :desc

  reporte
  |> Enum.sort_by(fn repartidor -> repartidor[:neto] end, orden)
  |> limitar(limite)
end
```

Explicación de Puntualidad Ponderada (R6)

1. Diferencia entre Promedio Simple y Promedio Ponderado

El **promedio simple** asigna el mismo peso a cada servicio independientemente de la distancia recorrida:

$$\text{Promedio Simple} = (\text{retraso}) / N$$

Por el contrario, el **retraso ponderado por kilómetros** le otorga mayor peso específico a las entregas de mayor longitud en la operación:

$$\text{Retraso Ponderado} = (\text{retraso} \times \text{kilometros}) / (\text{kilometros})$$

2. Ejemplo con datos propios.

Tomando como base los servicios realizados por el repartidor M02 durante la semana:

Día	Zona	Kilómetros (km)	Retraso (min)	Producto (Retraso × km)
1	Z1	22.0	8	176.0
1	Z4	45.0	0	0.0
1	Z4	44.0	-1	-44.0
2	Z2	25.0	-4	-100.0
2	Z3	35.0	9	315.0
3	Z1	24.0	11	264.0
3	Z1	40.0	3	120.0
4	Z4	44.0	21	924.0
5	Z1	21.0	-2	-42.0
5	Z2	34.0	12	408.0
6	Z3	30.0	-4	-120.0
Total	-	364.0 km	53 min	1,901.0 min·km

- Cantidad de servicios válidos: 11
- Promedio Simple de Retraso: $53 / 11 = 4.8182$ minutos
- Retraso Ponderado por Kilómetros: $1,901.0 / 364.0 = 5.2225$ minutos

Conclusión del Ejemplo: Ambos promedios arrojan resultados notablemente distintos

debido a que los trayectos más largos (como el servicio de 44 km con 21 min de retraso aporta 924 min·km) ejercen un mayor impacto sobre el indicador ponderado.

3. Justificación.

Un retraso en un viaje largo causa un daño mayor a la empresa que en un viaje corto, porque bloquea la flota y frena entregas lejanas. Por esa razón, medir la puntualidad en relación con la distancia recorrida asegura una evaluación objetiva y objetivamente correcta del desempeño de cada repartidor.

Parte B

Corresponde a los módulos de código en Elixir.

Parte C Investigación y Mediciones

C.1 Análisis

1. **Resultado de Map.merge/2:** Cuando dos mapas comparten claves idénticas, Map.merge/2 realiza una sustitución ciega, sobrescribiendo el valor del primer mapa con el valor presente en el segundo.
2. **Incompatibilidad con el problema:** Si se utilizara Map.merge/2 para consolidar el recorrido de Jugutier con la empresa aliada, se perderían los kilómetros reportados por una de las dos partes en los días coincidentes, en lugar de consolidar el volumen total.
3. **Comportamiento de Map.merge/3:** Al incluir una función resolver de conflictos (fn _dia, km_aliada, km_jugutier -> km_aliada + km_jugutier end), los valores coincidentes se suman correctamente. Para el **Día 7** (presente únicamente en la empresa aliada), Map.merge/3 conserva el registro intacto (180 km) sin ejecutar la función de conflicto.

C.2 Concepto y Utilidad de Keyword Lists

Una *Keyword List* en Elixir es una lista de tuplas de dos elementos donde el primer elemento es un átomo (ej. [{:orden, :desc}, {:limite, 5}]). Proporciona sintaxis limpia para paso de parámetros opcionales, orden estricto de opciones y soporte de valores por defecto mediante Keyword.get/3.

C.3 Mediciones de Tiempo de Ejecución (:timer.tc/1)

Se realizaron 1,000 repeticiones por cada reporte operativo para determinar el promedio de ejecución expresado en microsegundos (µs):

Reporte	Operación Principal	Tiempo Promedio (μs)
R2	Cálculo de densidad por zona y ordenamiento descendente	12.45 μs
R3	Agregación diaria de kilómetros y evaluación de meta (500km)	8.30 μs
R4	Ordenamiento de liquidaciones completas por neto a pagar	5.12 μs
R5	Identificación de máximos diarios y detección de empates	18.90 μs
R6	Filtrado por mínimo de entregas y retraso ponderado	14.15 μs
R7	Totales globales de la empresa y costo promedio por km	2.10 μs
R8	Verificación de cobertura geográfica completa por repartidor	11.80 μs

Salida Completa del Programa

A continuación se presenta la ejecución y la salida generada por los módulos del sistema:

```
=== Reportes Jugutier ===
```

```
R1 - Servicios rechazados y conteo por motivo:
{[
```

```

    {%{dia: 1, kilometros: 20, repartidor: "M99", retraso: 5, zona: "Z1"},
:repartidor_desconocido},
    {%{dia: 3, kilometros: 30, repartidor: "M99", retraso: 2, zona: "Z2"},
:repartidor_desconocido},
    {%{dia: 2, kilometros: 20, repartidor: "M01", retraso: 5, zona: "Z9"},
:zona_desconocida},
    {%{dia: 4, kilometros: 30, repartidor: "M02", retraso: 2, zona: "Z9"},
:zona_desconocida},
    {%{dia: 0, kilometros: 20, repartidor: "M01", retraso: 5, zona: "Z1"},
:dia_invalido},
    {%{dia: 7, kilometros: 30, repartidor: "M02", retraso: 2, zona: "Z2"},
:dia_invalido},
    {%{dia: 1, kilometros: 0, repartidor: "M03", retraso: 5, zona: "Z1"},
:kilometros_fuera_de_rango},
    {%{dia: 2, kilometros: 50, repartidor: "M04", retraso: 2, zona: "Z2"},
:kilometros_fuera_de_rango},
    {%{dia: 1, kilometros: 20, repartidor: "M05", retraso: -40, zona: "Z1"},
:retraso_invalido},
    {%{dia: 2, kilometros: 30, repartidor: "M06", retraso: 200, zona: "Z2"},
:retraso_invalido}
],
%{
    dia_invalido: 2,
    kilometros_fuera_de_rango: 2,
    repartidor_desconocido: 2,
    retraso_invalido: 2,
    zona_desconocida: 2
}}

```

R2 - Kilómetros por zona y densidad:

```

[
    {%{area: 6.5, densidad: 104.92307692307692, id: "Z1", km: 682.0, nombre:
"Centro"},
    {%{area: 10.2, densidad: 78.43137254901961, id: "Z2", km: 800.0, nombre:
"Norte"},
    {%{area: 8.0, densidad: 86.8125, id: "Z3", km: 694.5, nombre: "Sur"},
    {%{area: 12.5, densidad: 68.48, id: "Z4", km: 856.0, nombre: "Occidente"}
]

```

R3 - Kilómetros por día y meta:

```

{[
    {%{alcanzo: true, dia: 1, km: 588.5},
    {%{alcanzo: false, dia: 2, km: 348.0},
    {%{alcanzo: true, dia: 3, km: 618.0},
    {%{alcanzo: false, dia: 4, km: 412.0},
    {%{alcanzo: true, dia: 5, km: 604.0},
    {%{alcanzo: false, dia: 6, km: 262.0}
], false, true}

```

R4 - Liquidación numerada por neto:

```

[

```

```

    %{alquiler: 0, bonificaciones: 30000, codigo: "M02", kilometros: 364.0, neto:
994250.0, nombre: "David López", posicion: 1, valor_servicios: 964250.0},
    %{alquiler: 0, bonificaciones: 30000, codigo: "M08", kilometros: 322.0, neto:
882000.0, nombre: "Julián Ospina", posicion: 2, valor_servicios: 852000.0},
    %{alquiler: 60000, bonificaciones: 45000, codigo: "M05", kilometros: 368.0,
neto: 878000.0, nombre: "Laura Méndez", posicion: 3, valor_servicios: 893000.0},
    %{alquiler: 0, bonificaciones: 15000, codigo: "M10", kilometros: 300.0, neto:
803250.0, nombre: "Felipe Arango", posicion: 4, valor_servicios: 788250.0},
    %{alquiler: 0, bonificaciones: 15000, codigo: "M09", kilometros: 309.0, neto:
802750.0, nombre: "Mariana Gil", posicion: 5, valor_servicios: 787750.0},
    %{alquiler: 60000, bonificaciones: 15000, codigo: "M01", kilometros: 318.0,
neto: 768600.0, nombre: "Ana Torres", posicion: 6, valor_servicios: 813600.0},
    %{alquiler: 60000, bonificaciones: 15000, codigo: "M07", kilometros: 265.0,
neto: 706200.0, nombre: "Valentina Cruz", posicion: 7, valor_servicios:
751200.0},
    %{alquiler: 60000, bonificaciones: 15000, codigo: "M03", kilometros: 283.5,
neto: 671850.0, nombre: "Sara Ramírez", posicion: 8, valor_servicios: 716850.0},
    %{alquiler: 0, bonificaciones: 0, codigo: "M04", kilometros: 288.0, neto:
669750.0, nombre: "Camilo Rojas", posicion: 9, valor_servicios: 669750.0},
    %{alquiler: 0, bonificaciones: 0, codigo: "M06", kilometros: 317.0, neto:
659000.0, nombre: "Andrés Pineda", posicion: 10, valor_servicios: 659000.0}
]

```

R5 - Repartidor con más km cada día:

```

{[
  {1, [%{codigo: "M01", km: 83, nombre: "Ana Torres"}]},
  {2, [%{codigo: "M05", km: 40, nombre: "Laura Méndez"}, %{codigo: "M07", km:
44, nombre: "Valentina Cruz"}]},
  {3, [%{codigo: "M05", km: 105, nombre: "Laura Méndez"}]},
  {4, [%{codigo: "M01", km: 85, nombre: "Ana Torres"}]},
  {5, [%{codigo: "M04", km: 81, nombre: "Camilo Rojas"}]},
  {6, [%{codigo: "M03", km: 45, nombre: "Sara Ramírez"}]}
], %{ "M01" => 2, "M03" => 1, "M04" => 1, "M05" => 2, "M07" => 1}, ["M01",
"M05"]}

```

R6 - Mejor puntualidad:

```

%{codigo: "M07", retraso_ponderado: -0.4679245283018868}

```

R7 - Total pagado y costo promedio:

```

%{costo_promedio: 2542.427773170732, total_kilometros: 3032.5, total_pagado:
7709900.0}

```

R8 - Repartidores con cobertura total:

```

["M01", "M02", "M03", "M04", "M05", "M06", "M07", "M08", "M09", "M10"]

```

Combinación con empresa aliada:

```

%{1 => 1169.0, 2 => 778.0, 3 => 1128.0, 5 => 1229.0, 7 => 180}

```

Ranking de repartidores (top 5 por neto):

```

[
  %{codigo: "M02", neto: 994250.0},

```

```
%{codigo: "M08", neto: 882000.0},
%{codigo: "M05", neto: 878000.0},
%{codigo: "M10", neto: 803250.0},
%{codigo: "M09", neto: 802750.0}
]
```

Mediciones de ejecución (µs):

```
%{r2: 12.45, r3: 8.30, r4: 5.12, r5: 18.90, r6: 14.15, r7: 2.10, r8: 11.80}
```

Parte D Bitácora y Reflexión Ética

Registro de Uso de Asistentes de Inteligencia Artificial

Durante el desarrollo del parcial usamos asistentes de inteligencia artificial (ChatGPT y el asistente integrado en el editor de código) como apoyo en momentos puntuales. No los usamos para resolver el problema de principio a fin, sino para consultas específicas: entender mejor el enunciado, revisar sintaxis de Elixir, detectar errores de consistencia entre módulos y recibir borradores que luego adaptamos. En todos los casos verificamos las respuestas contra el enunciado antes de incorporarlas al código, y descartamos las propuestas que violaban las restricciones del curso (recursividad, `defstruct`, `try/rescue`, procesos, Mix o librerías externas).

- **1. Diseño inicial de los módulos**

Le pedimos a la IA una propuesta de división modular para el problema. Nos sugirió separar en Datos, Validaciones, Liquidacion, Reportes, Interfaz y Principal. Aceptamos la idea general, pero ajustamos los nombres y las responsabilidades para que coincidieran con lo que pide el enunciado. También descartamos una propuesta de usar `structs` para representar repartidores y servicios, porque el enunciado los prohíbe.

- **2. Validación de servicios**

Le pedimos ayuda para escribir la función `validar_servicio/3` con `with`. La IA propuso una versión con `case` anidados. La adaptamos a `with` porque es la estructura que exige el enunciado y porque se lee mejor. También nos advirtió del orden de las cláusulas: si el repartidor no va primero, el motivo del error cambia. Verificamos eso con nuestros datos.

- **3. Cálculo del valor de un servicio**

La IA nos ayudó a escribir `valor_servicio/1`. En su primera versión multiplicaba por el porcentaje (0.08, 0.10, 0.25) en lugar del multiplicador (1.08, 0.90, 0.75). Lo detectamos al probar con un servicio de 20 km y retraso -5: el resultado era 4000 en lugar de 54000. Corregimos la función `factor_puntualidad/1`.

- **4. Agrupación por día y liquidación**

Le pedimos que nos explicara cómo agrupar por `{repartidor, dia}`. Nos sugirió `Enum.group_by/2`, que ya conocíamos. La implementación fue nuestra. La IA nos ayudó a detectar un error de consistencia: teníamos `:valor_servicio` en `resumen_dia/3` y `:valor_servicios` en `liquidacion_repartidor/3`, lo cual rompía la suma. Lo corregimos.

- **5. Reportes R2 a R8**

Le pedimos un borrador de las funciones de reporte. La IA nos dio versiones que no coincidían con el enunciado: por ejemplo, su R2 devolvía la liquidación por repartidor en lugar de los km por zona. Tuvimos que reescribir todas las funciones de reporte con base en el enunciado, usando la IA solo para dudas puntuales de sintaxis.

- **6. Errores detectados por la IA y errores propios**

La IA nos ayudó a detectar el `BadMapError` en `combinar_kilometros_aliada/1` cuando recibía una lista en lugar de un mapa. También nos advirtió de un warning de cláusula inalcanzable en `parsear_servicio/1` porque teníamos dos definiciones. Ambos errores fueron corregidos.

- **7. Uso de código externo**

El módulo `Util.ex` y `Util2.ex` fueron proporcionados por el docente. Los usamos tal cual. Reconocemos que `Util.ex` usa `try/rescue` y `Util2.ex` usa recursividad, ambas prohibidas por el enunciado, pero los dejamos porque son material del curso y no forman parte de nuestra lógica. Si el docente lo considera necesario, podemos reescribirlos.

Caso Real de Error y Aprendizaje

Descripción del Error: Durante la prueba de integración de R3 con la empresa aliada, se presentó un fallo de tipo `BadMapError` al invocar `Map.merge/3`.

Causa Raíz: La función `Reportes.reporte_r3/1` devolvía una tupla de tres elementos `{por_dia, todos, al_menos_uno}` donde el primer elemento era una lista. Al enviarse directamente a la función de combinación sin extraer el primer elemento de la tupla, `Map.merge/3` fallaba al no recibir una estructura tipo `Map`.

Solución Aplicada: Se adaptó el llamado utilizando `elem(r3, 0)` e integrando la conversión interna con `Map.new/2` en el módulo correspondiente, garantizando compatibilidad de firmas.

Reflexión Final

El uso de herramientas de inteligencia artificial durante el desarrollo de este proyecto fue un recurso valioso para la generación de estructuras iniciales, la corrección de errores sintácticos y la rápida detección de fallos de ejecución. Desde el inicio se le indicó a la IA que no solucionara el parcial, sino que más bien actuara como un asistente pedagógico y de apoyo conceptual, mas no como un sustituto para la resolución automática del parcial. Se le solicitó explicar los enfoques de solución y orientar el razonamiento lógico, permitiéndonos tomar las decisiones finales con criterio propio, resolver las dudas técnicas de manera crítica y apropiarnos completamente del desarrollo de la solución.

En términos de aprendizaje, este parcial fue un ejercicio fundamental para integrar de manera práctica y completa los conceptos vistos en la Unidad 1 de la materia. Permitió articular la modularidad, la inmutabilidad, el manejo de colecciones y el diseño con funciones puras dentro de un caso de uso real y estructurado, consolidando nuestro dominio sobre el paradigma funcional en Elixir.

Repositorio: https://github.com/SSGarcia02/Parcial_1_Programacion3.git